

Ask, Then Think: Enhancing LLM Performance with Socratic Reasoning

Antonio Della Porta
adellaporta@unisa.it
University of Salerno
Salerno, Italy

Jonan Richards
jonan.richards@ru.nl
Radboud University
Nijmegen, The Netherlands

Lucageneroso Cammarota
l.cammarota8@studenti.unisa.it
University of Salerno
Salerno, Italy

Stefano Lambiase
stla@cs.aau.dk
Aalborg University
Copenhagen, Denmark

Fabio Palomba
fpalomba@unisa.it
University of Salerno
Salerno, Italy

Mairieli Wessel
mairieli.wessel@ru.nl
Radboud University
Nijmegen, The Netherlands

Abstract

Large Language Models (LLMs) are widely used for code generation, repair, and explanation, yet they still make mistakes when instructions are vague or when their initial assumptions are wrong. Current prompting methods help models show their reasoning but do not encourage them to check whether that reasoning is sound. This *confirmatory* study introduces *MIND*, a Socratic-style prompting framework that guides models through self-questions aimed at clarifying the task, identifying missing information, and checking intermediate conclusions. We will evaluate the framework across four benchmarks covering code generation, repair, explanation, and efficiency, comparing it with methods such as Chain-of-Thought, Self-Ask, Self-Refine, and ReAct. We will also run an ablation study to examine which stages of the questioning process contribute most to performance.

Keywords

Prompt Patterns, Cognitive Theories, Source Code Quality

1 Introduction

Large Language Models (LLMs) have become widely used in software engineering for problem solving [21], code generation [13], and supporting reasoning tasks [11]. As their use spreads, expectations rise, and users increasingly want systems that show clear understanding, cope with unclear instructions, and avoid errors caused by false assumptions [7]. Yet current models still show clear limitations when instructions are vague or when essential context is missing [24].

A substantial amount of work has aimed to improve this through prompting methods. Techniques like Chain-of-Thought [22] and ReAct [25] show that asking a model to spell out its reasoning can help, but these methods mostly guide the model to think in a straight, predictable way. The nature of these prompting strategies leads the model to define and perform steps, but not to question whether those steps make sense in the first place or if they are going in the correct direction to answer the user’s request [17]. As a result,

the model can produce detailed step planning that can potentially miss the point when the original problem was misunderstood.

We as humans are naturally inclined to create an initial plan for solving a problem and then strengthen it by looking for flaws or missing conditions [9]. For example, a developer might sketch out an algorithm, realize that an edge case breaks the logic, and then adjust the plan before implementing it. The main idea behind this work is to bring a human-like reasoning process to LLMs’ reasoning process.

To this aim we introduce *MAIEUTIC INFERENCE AND DISCOVERY (MIND)*, a prompting framework inspired by classical Socratic reasoning [3, 8, 20] that is based on the Socratic theory of thinking. Instead of simply asking the model to think step by step, *MIND* leads the model through short, focused self-questions that help it clarify the task, identify missing information, break the problem into parts, and check whether the final answer makes sense. The goal is to bring a small amount of structured self-interrogation into the reasoning process. While existing prompting methods encourage models to show their chain of thought, *MIND* pushes them to examine it by spotting vagueness, checking assumptions, identifying constraints, and challenging intermediate conclusions. In this way, it adds a form of internal quality control that other prompting strategies usually don’t have.

To study whether this approach actually helps, we test it across four code-related benchmarks: BigCodeBench [26], HumanEvalFix and HumanEvalExplain [15], and Mercury [6]. These benchmarks cover a wide range of tasks, from writing code and fixing bugs to understanding program behavior and optimizing performance. Along with measuring how well the model performs, we also run a step-by-step ablation study to understand which parts of this self-questioning process are the most impactful on the entire process, in order to give insights to the research community on which steps had impact on the overall performance, and also where to improve.

Our aim is to assess whether LLMs benefit from a prompting approach that encourages more deliberate reasoning instead of surface pattern completion. If the approach improves performance, it would indicate that prompting can be used not only to elicit reasoning traces but also to influence the quality of the reasoning process itself.

Authors’ Contact Information: Antonio Della Porta, adellaporta@unisa.it, University of Salerno, Salerno, Italy; Jonan Richards, jonan.richards@ru.nl, Radboud University, Nijmegen, The Netherlands; Lucageneroso Cammarota, l.cammarota8@studenti.unisa.it, University of Salerno, Salerno, Italy; Stefano Lambiase, stla@cs.aau.dk, Aalborg University, Copenhagen, Denmark; Fabio Palomba, fpalomba@unisa.it, University of Salerno, Salerno, Italy; Mairieli Wessel, mairieli.wessel@ru.nl, Radboud University, Nijmegen, The Netherlands.

2 Related Works

Several studies have used structured reasoning strategies for different software engineering tasks [17].

Among the first contributions in reasoning-based prompt patterns, we find Chain-of-Thought proposed by Wei et al. [22] that prompts models to produce explicit intermediate steps before giving the final answer. This often improves accuracy by encouraging more detailed task decomposition, but the steps are generated in a purely forward manner. The model explains its reasoning without being required to question the validity of those steps, identify hidden assumptions, or test whether its partial conclusions hold.

Madaan et al. [14] introduce Self-Refine, which asks the model to generate an initial solution, produce feedback on its own output, and then revise it accordingly. The refinement loop can improve clarity and correctness, but the feedback focuses on editing the written result rather than interrogating the underlying reasoning. As a consequence, the process revises outputs without enforcing a deeper examination of assumptions or logical soundness.

Yao et al. proposed ReAct [25], a prompting strategy that alternates brief reasoning with actions like tool calls or searches, allowing the model to adapt based on observations. Its reasoning is oriented toward choosing the next action, not evaluating assumptions or verifying intermediate conclusions, which limits its ability to support deeper self-checking.

Press et al. introduced Self-Ask [18], a prompting process that follows a reasoning process guided by a sequence of explicit sub-questions that the model answers one by one. This helps break complex tasks into smaller units, but the process remains strictly additive. The model generates sub-questions and answers them without examining whether the questions are well-posed, whether assumptions are correct, or whether its intermediate reasoning contains contradictions. As a result, Self-Ask improves organization but does not enforce deeper self-interrogation.

Compared to these patterns, *MIND* shifts the target of structure from the surface form of the reasoning to the checks the model is required to perform on its own thinking. Chain-of-Thought, Self-Refine, ReAct, and Self-Ask all prescribe how to expand, refine, or operationalize a solution: they tell the model to write intermediate steps, to revise its output, to alternate thoughts and actions, or to answer sub-questions. In all cases, the reasoning trace is a by-product of producing an answer. It is not treated as an object that must itself be examined for missing premises, unjustified leaps, or unresolved conflicts. At each stage, the model is prompted to clarify what the problem really asks, to make its assumptions explicit, to check whether they are warranted by the prompt, to search for counterexamples to its current plan, and to reconcile any issues it uncovers before proceeding. The framework, therefore, does not simply lengthen or reorganize the chain of thought. It turns the reasoning process into a succession of challenges to the model's own claims, with the explicit aim of rejecting or repairing flawed lines of thought. This structure is what differentiates our approach from existing prompting patterns that improve organization and interaction, but leave the validity of the reasoning largely unchecked.

Research Gap

Existing prompting strategies add structure to reasoning but **do not require the model to examine whether its assumptions, intermediate steps, or conclusions are valid.** They improve organization and detail, yet leave the **soundness** of the reasoning unchecked. The gap lies in the absence of **methods that enforce explicit self-interrogation within the reasoning process.**

3 Maieutic Inference and Discovery approach

Recent studies in the AI field [1, 5, 10] shows that LLMs display content-dependent reasoning biases and are able to internally represent their own knowledge and uncertainty. Based on these findings on similarities between humans' and LLMs' thought processes, we aim to extend the concept in the SE field to further evaluate the extent to which processes defined to improve human reasoning can be applied to improve the performance of LLMs.

The approach we will evaluate is based on the classical *Socratic reasoning* process, a form of philosophical inquiry that examines beliefs through structured dialogue [3]. Such process starts from the idea that people often hold views that seem clear to them but contain hidden assumptions or unresolved tensions. By asking focused questions, Socrates leads the interlocutor to inspect the foundations of their thinking rather than receiving answers from an external authority.

The classical Socratic process used to build the proposed approach is composed of the following three main phases [20]:

Elenchos. In this first phase, the *Elenchos* works as a careful check of the initial claim. Socrates pushes the interlocutor to say clearly what the problem is, what assumptions they are relying on, and what follows from taking those assumptions seriously. This removes vagueness and makes hidden premises visible. Once everything is stated plainly, any clash with the interlocutor's other beliefs becomes obvious, and the initial claim often collapses. This collapse is useful because it shows which parts of the problem are solid, which are unclear, and which pieces of information or constraints are actually missing.

Aporia. When the initial claim falls apart, the interlocutor enters *Aporia*. The tensions exposed during the *elenchos* point directly to the parts that need to be examined one by one, and these become separate questions. The problem is broken into smaller, clearer components, each of which can be checked independently. This division is guided by the weak spots uncovered earlier, so it is neither random nor superficial. With the problem now decomposed, the interlocutor can explore each part, test where it might fail, and verify whether solving these parts genuinely solves the whole.

Maieutica. Once the key ideas are clarified and the problem is divided into stable parts, the *Maieutica* allows a complete answer to take shape. Socrates helps the interlocutor rebuild the solution by connecting the pieces in a clear and ordered way. He does not provide the answer himself; he encourages the interlocutor to follow the structure that emerged from their own reasoning. Because the earlier steps removed contradictions and validated the plan, the final idea appears as the natural outcome of the process. The conclusion is then checked against the original question to ensure it actually addresses it and does not introduce new issues.

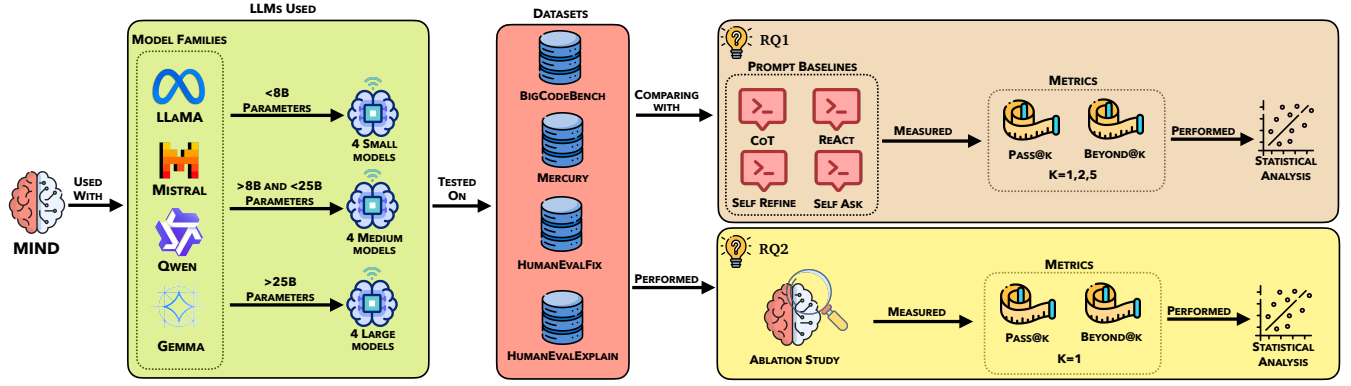


Figure 1: Overview of the Research Method

Based on this reasoning process, we propose the MAIEUTIC INFERENCE AND DISCOVERY (*MIND*) approach, which enables language models to reason inspired by the Socratic Maieutic method. We adapt this concept for language models not as a simple, one-shot prompt, but as a structured, multi-phase dialectic that emulates the process Socrates used with his students to teach how to tackle complex questions. In Table 1 we describe the reasoning process by mapping the classical Socratic method onto the same three phases, and we keep the goals of each phase close to how they work in the original practice.

To implement the approach, we will start by first building a complete prompt pipeline that encodes each phase of the method as a separate prompt. These prompts will then be tested on one model per size category to confirm that the structure is followed. We will check whether the model interprets each step correctly, whether the answers stay within the intended boundaries of the phase, and if there are missing sub-steps or skipped parts of the reasoning.

4 Method

The *goal* of the methodology we are proposing is to empirically evaluate the performances of the *MIND* approach compared with some baseline approaches in complex, code-related tasks, and also investigate which is the part that contributes the most to the outcomes of the *MIND* approach to improve our understanding of cognitive theories in LLMs and also to have some insights on what to improve. Following established guidelines for empirical study design [23], we now formulate two research questions that guide our investigation and help achieve the study’s objectives.

RQ₁ – To what extent does the *MIND* approach improve the performance of LLMs compared to established prompting baselines across standard benchmarks?

With this question, we aim to assess whether *MIND* indeed improves performance on diverse code-related tasks and whether it can be used in real-world scenarios. To answer this research question, we will quantitatively evaluate whether the *MIND* approach outperforms established reasoning-based methods such as Chain-of-Thought, Self-Refine, Self-Ask, and ReAct across different benchmarks, each targeting different programming activities. To

quantify the improvement, we will measure the functional correctness of the generated solutions using the task-specific tests of each benchmark and compute the *pass@k* and *Beyond@k* metric. We will also measure the inference time duration in all the experiments to weight the accuracy with the time needed to get a solution.

RQ₂ – Which components of the *MIND* approach contribute most significantly to its overall performance?

This research question examines the individual stages that compose the *MIND* approach. Although each stage is intended to clarify the problem and strengthen the reasoning process, their actual contribution may vary. The ablation study, therefore, serves two purposes. If RQ₁ reveals weaknesses or uneven gains, the ablations help identify which steps are responsible and where improvements are needed. If RQ₁ shows consistently strong performance, the ablations clarify which components are essential, which are redundant, and whether the approach can be simplified without losing effectiveness. To do this, we will evaluate ablated variants of *MIND* on the same benchmarks as RQ₁ and measure the performance changes associated with removing each component.

4.1 Data Extraction

4.1.1 LLMs Selection. To make sure that the performance of *MIND* is not tied to a specific model scale, we evaluate it across LLMs of different sizes and different model architectures.

For the size, we follow a simple size-based grouping commonly used in recent multi-model studies [26]: **small** models (up to 8B parameters), **medium** models (9B–25B), and **large** models (more than 25B). We cap the upper end at 70B parameters to keep the experiments feasible with our available compute.

For the architectures, our goal is to assemble a representative yet manageable set, so we focus on four widely used open-weight families which together cover the main model designs found in current research: **Mistral**, **Gemma**, **LLaMA**, and **Qwen**.

To build the set of models that will be used throughout our evaluation, we will adopt the following criteria: within each size category, we select one model from each family using the most recent publicly available checkpoint. When several candidates are

Table 1: Socratic Prompting Framework: Goals and Self-Interrogation Mechanics

Step	Goal	Questions Posed
Phase 1: Elenchos – Refutation & Definition		
1.1 Initial Refutation	Eliminate ambiguity and define exact problem scope.	Where exactly is this request vague? What hidden assumption is the user making that I must challenge or clarify to avoid failure?
1.2 Premise Seeking	Identify certain knowledge, standard tools, and undeniable truths.	What do I know for certain? What libraries or algorithms can serve as a secure base?
1.3 Aporia Discovery	Identify missing information and hard constraints.	What is the core piece of missing knowledge? What is the constraint that, if violated, invalidates everything?
Phase 2: Aporia – Division & Planning		
2.1 Decomposition	Divide the complex problem into manageable, logical parts.	Can this problem be divided into smaller parts? Is this division exhaustive?
2.2 Sub-Problem Inquiry	Stress-test sub-problems to find breaking points (edge cases).	How could this specific part fail? What input would falsify my current hypothesis for this sub-part?
2.3 Path Validation	Validate logical sequence; ensure parts sum to the whole.	If I solve all the sub-problems, is the whole definitely solved? Is the execution order logically unassailable?
Phase 3: Maieutica – Birthing the Solution		
3.1 Solution Crafting	Synthesize final solution strictly following the validated plan.	Execute the full plan that was defined. Take the sub-problems in sequence, generate the solution for each, and then create the final response.
3.2 Final Validation	Validate final solution against original intent, not just the plan.	Does this satisfy the <i>original</i> request? Have I introduced new, unnecessary contradictions?

available (e.g. LLaMA 3.2 1B and 3B in the small cluster), we pick the largest one to better represent the family’s capabilities.

The list of selected models is reported in Table 2, but it may be revised to incorporate for model deprecations. The replacement of a model will only take place if, when executing the study, the developer has deprecated the model chosen, and in such case, we will use (1) the replacement model that the developers eventually published if it lies in the same size category as the deprecated one, or (2) the latest available model in the size category of the deprecated model.

All the LLMs used will be distributed as GGUF checkpoints, using Q4 quantized variants. Decoding will use temperature = 0.7. All other decoding and inference parameters will be left at the default settings of the selected GGUF inference engine, and we will report the exact model version used in the study.

Following empirical guidelines for LLM-based studies in software engineering proposed by Baltes et al. [2], all experiments will include for each model used, its fingerprint information to ensure reproducibility and traceability. Moreover, experiments will be conducted under consistent conditions, using the same hyperparameters across models: the temperature will be fixed at 0.8 and the top-p parameter at 0.95. These values were selected based on their adoption in similar settings on code-related tasks[12].

4.1.2 Baseline Selection. To have a clear picture of the *MIND* approach performances, we will compare it against prompting techniques that also rely on structured or self-reflective reasoning. We

Table 2: LLMs used in the study

Small		Medium		Large	
Name	Size	Name	Size	Name	Size
LLaMA 3.2	3B	LLaMA 3.1	13B	LLaMA 3.3	70B
Minstral	8B	Mistral Small 3.2	24B	Mixtral	42B
Qwen 3	8B	Qwen 3	14B	Qwen 3	30B
Gemma 3	4B	Gemma 3	12B	Gemma 3	27B

select well-established baselines from the literature[16] that follow this style: **Chain-of-Thought** [22], **Self-Ask** [18], **ReAct** [25], and **Self-Refine** [14]. Chain-of-Thought represents step-by-step deductive reasoning and serves as a minimal structured prompting baseline. Self-Ask decomposes a problem into explicit sub-questions, encouraging a more modular form of reasoning. ReAct integrates reasoning and action by alternating between intermediate reasoning steps and tool or environment interactions, testing how reasoning emerges through feedback. Finally, Self-Refine introduces iterative self-correction, where the model revises its own outputs through feedback loops.

To minimize variability due to prompt writing, we will use the prompt templates used in the papers that originally proposed these approaches and on which the original evaluation was performed.

Table 3: Datasets used in the study

Name	Task Performed	#Problems
BigCodeBench [26]	Code Generation	1140
Mercury [6]	Code Efficiency	256
HumanEvalFix [15]	Code Repair	164
HumanEvalExplain [15]	Code Explanation	164

The prompts that we will use will be shared in the replication package.

4.1.3 Benchmarks Selection. To conduct our experimentation, we selected four public benchmarks that aim at measuring the capabilities of models in code-related tasks, listed in Table 3. These datasets cover complementary aspects of coding tasks, including code generation, optimization, repair, and explanation, allowing a comprehensive assessment of both the productive and reflective capabilities that the *MIND* approach aims to enhance.

BigCodeBench [26] provides a large set of code generation tasks and serves as the main reference for evaluating how effectively a model can compose functional solutions from natural-language problem descriptions.

Mercury [6] focuses on code efficiency and evaluates whether models can produce code solutions that are not only functionally correct but also efficient.

HumanEvalFix and HumanEvalExplain [15] extend the original HumanEval [4] benchmark to the domains of program repair and code explanation, testing the model’s ability to identify and correct errors or to describe the purpose and behavior of a code snippet.

4.2 Data Collection

To answer the research questions, we will begin by collecting the performance of the *MIND* approach and the selected baseline approaches on the four selected benchmarks. For each benchmark, we will develop an automated pipeline that performs the task and outputs all relevant data, such as the generated solutions, error codes, and reasoning traces, to an external file for transparency.

For **BigCodeBench** [26], the pipeline will prompt models with the natural language problem description and make each of the selected models, with each of the selected baselines, generate the code that represents the solution described. The generated code will then be tested with the official test suite for each problem that is provided by the dataset. The metric used to assess the performances in this benchmark will be the *pass@k* with $k \in \{1, 2, 5\}$, that will quantify the functional correctness of the generated code in up to 5 different generations.

For **HumanEvalFix** and **HumanEvalExplain** [15], we will follow the procedures defined in the original article that proposed the benchmarks. In **HumanEvalFix**, each model will be provided with a buggy implementation and the corresponding problem statement, and the task they will perform is to fix the buggy code and make it pass the test cases. The metric used to assess the performances in this benchmark will be the *pass@k* with $k \in \{1, 2, 5\}$, that will evaluate whether the model was able to fix the code snippet having up to 5 different attempts. For **HumanEvalExplain**, the process

will be split into two separate steps. In the first step, each model will be provided with a code snippet and will first be asked to produce a natural language explanation of the snippet. The second step consists of taking the natural language description and then, in a separate stateless API call, the model will be asked to regenerate the code from that explanation. The *pass@k* metric will evaluate whether the model was able to understand the content of the code snippet in up to 5 different attempts.

For **Mercury** [6], models will be prompted to generate code solutions from natural language problem statements. The generated solutions will be evaluated using the *Beyond* metric, the same that is proposed in the original article to evaluate the performances on the Mercury benchmark. The *Beyond* metric measures runtime efficiency by computing the percentile of the generated solution’s execution time within the runtime distribution of verified human-written solutions defined in the benchmark, assigning higher scores to code that is more computationally efficient. In our case, we will use an adapted version of the *Beyond* metric that we called *Beyond@k* with $k \in \{1, 2, 5\}$. Since each model produces five candidate solutions as in the other benchmark tasks, we compute *Beyond@k* by selecting, respectively, the best score among the top 1, top 2, and top 5 generated solutions.

All the executions of these experiments will be timed using Python’s built-in `time` library. Specifically, for each LLM call we will measure the delta time (in milliseconds) between the timestamp captured immediately before issuing the inference request and the timestamp captured immediately after the response is fully received.

4.3 Data Analysis

To answer RQ₁, the performance of the *MIND* approach will be compared against all selected baselines using the metrics defined by each benchmark, and will also take into account the different families of models, their size, and the specific task (e.g., code efficiency, code repair) that the LLM has performed. On all the problems that are in BigCodeBench, HumanEvalFix, and HumanEvalExplain, we will compute *pass@k* metric with $k \in \{1, 2, 5\}$, and for the problems in the Mercury dataset, we will compute the *Beyond@k* metric with the same values of k .

The dependent variable of the analysis will be the aforementioned *pass@k* and *Beyond@k* metrics, while for the independent variables, we will group the observations using the type of **prompting strategy** used (e.g., *MIND* and *ReAct*), while also taking into account the **model family**, the **size** cluster it was in, and the **task** that was performed.

The hypotheses that will guide the statistical analysis will be the following:

- **Null Hypothesis:** There are no significant differences in the evaluation metrics based on
 - ($H1_0$) The prompting strategy used,
 - ($H2_0$) The model family,
 - ($H3_0$) The model size cluster,
 - ($H4_0$) The task type.
- **Alternative Hypothesis:** There are significant differences in the evaluation metrics based on
 - ($H1_a$) The prompting strategy used,

- ($H2_a$) The model family,
- ($H3_a$) The model size cluster,
- ($H4_a$) The task type.

Given that $pass@k$ and $Beyond@k$ are bounded proportions in $[0, 1]$, and that we will have multiple observations of the same problem, we will model the analysis using **beta mixed-effects regression**. Before fitting the model, we will (1) check model assumptions (e.g., adequacy of the beta distribution and random-effects structure) through standard residual and convergence metrics, and (2) rescale the values to avoid boundary issues, since the outcome must be restricted to the open interval $(0, 1)$. To this aim, we will follow Smithson and Verkuilen [19] to apply a standard transformation that maps values into the open interval $(0, 1)$.

If the assumptions of the model are violated or the models fail to converge, we will fall back on non-parametric alternatives for repeated measures, such as the Friedman test or aligned-rank transformation ANOVA applied to the same metrics. For statistically significant effects, we will perform post-hoc pairwise comparisons with Holm–Bonferroni correction for multiple testing and report effect sizes based on contrasts of estimated marginal means on the link scale (e.g., odds ratios or standardized differences), together with confidence intervals.

To address efficiency concerns, we additionally track inference duration for every run and report it as a descriptive measure of overhead. The goal is to make the trade-off between effectiveness and efficiency explicit. Concretely, for each model family and size, we summarize the inference-time distributions of MIND and the baseline prompting strategies using quartiles, and we report (i) the relative change in inference time of MIND compared to each baseline and (ii) differences in interquartile range (IQR) to capture changes in dispersion. This comparison of inference times will allow us to weigh the potential benefits in accuracy of the MIND approach against the potential efficiency losses.

To answer RQ₂, we will investigate the contribution of each component of the MIND framework using a single-step ablation study. Since the approach consists of eight steps (1.1 to 3.2), we will construct ablated variants by removing each step individually, except for Step 3.1 (Solution Crafting), which cannot be disabled without preventing solution generation. Each ablated variant, therefore, removes exactly one of the remaining seven steps while keeping all others unchanged. All ablations will be performed under the same conditions as the full MIND approach, using identical inputs, decoding settings, and evaluation pipelines.

Ablations will be evaluated with $k = 1$, using either $pass@1$ or $Beyond@1$ depending on the benchmark. The impact of each component will be characterized through the absolute and relative changes in performance with respect to the full MIND system. We will summarize the observed trends using descriptive statistics and highlight the configurations that produce the largest changes, providing an interpretable view of how each module influences overall effectiveness.

To reduce ambiguity in interpreting the outcomes, we pre-declare a hierarchy of effects and how conclusions will be framed. The primary (confirmatory) target of inference is the overall effect of the prompting strategy, i.e., whether MIND improves performance relative to the baselines when aggregating across all tasks, benchmarks,

and model configurations under the pre-registered mixed-effects model. Our main conclusion about the effectiveness of MIND will be based on this primary effect, rather than on isolated positive slices. All task-, benchmark-, and model-specific effects (including interactions such as prompting strategy $\times$ task type or prompting strategy $\times$ model size) are treated as secondary analyses and will be interpreted as evidence of heterogeneity or moderators. Accordingly, if the primary effect is null or practically negligible but improvements appear only for a specific benchmark, task, or model category, we will report these as conditional, task-scoped findings and refrain from claiming overall superiority, using them instead to motivate targeted follow-up investigations.

5 Discussions

The final goal of this work is to investigate whether a structured prompting pipeline can make LLM behavior more reliable and explainable for SE work by explicitly enforcing clarification, assumption checking, and validation steps, rather than relying on a single one-shot prompt. In our case, we built our reasoning framework on the Socratic approach to thinking, drawing on prior literature showing that LLMs exhibit content-dependent reasoning biases and can internally represent their own knowledge and uncertainty [1, 5, 10]. Moreover, MIND operationalizes explainability as process transparency: it produces an explicit, step-wise reasoning trace that separates (i) interpreted requirements, (ii) stated assumptions and unknowns, (iii) intermediate plans or candidate solutions, and (iv) validation outcomes. This enables attribution of failures to specific stages (e.g., misread requirements vs. incorrect implementation) and to audit when the model proceeded despite unresolved uncertainty.

The relevance of the study for SE is given by the type of evidence it is designed to produce. We evaluate MIND against established reasoning baselines (Chain-of-Thought, Self-Ask, ReAct, Self-Refine) using four benchmarks that represent different programming activities: code generation (BigCodeBench), repair (HumanEvalFix), explanation (HumanEvalExplain), and efficiency (Mercury). This matters because SE practice is not a single task, and prompting methods that perform well in one setting may not transfer to others. The protocol also measures both effectiveness and inference time, enabling the study to assess practical trade-offs that determine whether an approach is usable in real-world tools. In addition, the structured traces produced by MIND provide SE-relevant explanatory artifacts: they can be inspected by developers, logged by tools, and compared across runs, which supports debugging and accountability beyond aggregate accuracy. By contrast, baselines may yield correct answers without revealing which assumptions or checks drove the outcome, thereby limiting their usefulness when diagnosing integration failures.

The findings will be useful to SE in two ways. First, they will assess whether structured self-questioning yields a general improvement across SE tasks or primarily helps in specific contexts. For example, if gains concentrate in repair and explanation rather than generation, that would align with the broader observation that performance depends on task framing and context, and it would suggest deploying MIND-like checks primarily where misunderstanding requirements is the dominant failure mode. Second, the ablation study is designed to identify which MIND steps contribute

to performance, thereby informing lighter, more practical variants for IDE assistants and automation pipelines.

6 Threats to Validity

Internal validity. A primary concern lies in the variability introduced by stochastic generation in LLM outputs. To mitigate this, all models are evaluated under fixed decoding parameters and each prompting condition is applied independently across all models and benchmarks. Because pass@k and Beyond@k aggregate multiple samples, they reduce but do not eliminate stochastic noise. Another source of internal threat is the correctness of the benchmark execution pipelines. We address this by using the official test suites provided by the datasets and by exporting raw model outputs and execution logs to allow post-hoc inspection.

External validity. Our selection of four benchmark families offers broad coverage of code-related tasks but cannot represent all programming domains or prompt scenarios. Likewise, we include multiple LLM families and sizes to improve generalizability, yet the results may not transfer to proprietary models, multimodal models, or models trained with different architectural priors. Similarly, the upper bound of 70B parameters is dictated by resource constraints, and larger-scale models could exhibit qualitatively different behaviour. Ablation effects might also depend on model scale in ways not fully captured by the three size categories we employ.

Conclusion validity. The use of standard assumption checks and mixed-effects models mitigates risks of inappropriate statistical inference, but Type I and Type II errors remain possible given the large number of comparisons across models, benchmarks, baselines, and ablations. We address this through Holm–Bonferroni correction and by reporting effect sizes with confidence intervals. However, statistical significance may not always align with practical significance, especially in tasks with small absolute performance differences or high variance in model outputs.

References

- [1] ANTHROPIC. Code execution with mcp: Building more efficient agents. <https://www.anthropic.com/engineering/code-execution-with-mcp>, 2025.
- [2] BALTES, S., ANGERMEIR, F., ARORA, C., BARÓN, M. M., CHEN, C., BÖHME, L., CALEFATO, F., ERNST, N., FALESSI, D., FITZGERALD, B., ET AL. Guidelines for empirical studies in software engineering involving large language models. *arXiv preprint arXiv:2508.15503* (2025).
- [3] BENSON, H. H. *Socratic wisdom: the model of knowledge in Plato's early dialogues*. Oxford University Press, 2000.
- [4] CHEN, M., TWOREK, J., JUN, H., YUAN, Q., DE OLIVEIRA PINTO, H. P., KAPLAN, J., EDWARDS, H., BURDA, Y., JOSEPH, N., BROCKMAN, G., RAY, A., PURI, R., KRUEGER, G., PETROV, M., KHLAUF, H., SASTRY, G., MISHKIN, P., CHAN, B., GRAY, S., RYDER, N., PAVLOV, M., POWER, A., KAISER, L., BAVARIAN, M., WINTER, C., TILLET, P., SUCH, F. P., CUMMINGS, D., PLAPPERT, M., CHANTZIS, F., BARNES, E., HERBERT-VOSS, A., GUSS, W. H., NICHOL, A., PAINO, A., TEZAK, N., TANG, J., BABUSCHKIN, I., BALAJI, S., JAIN, S., SAUNDERS, W., HESSE, C., CARR, A. N., LEIKE, J., ACHIAM, J., MISRA, V., MORIKAWA, E., RADFORD, A., KNIGHT, M., BRUNDAGE, M., MURATI, M., MAYER, K., WELINDER, P., MCGREW, B., AMODEI, D., MCCANDLISH, S., SUTSKEVER, I., AND ZAREMBA, W. Evaluating large language models trained on code, 2021.
- [5] DASGUPTA, I., LAMPINEN, A. K., CHAN, S. C., CRESWELL, A., KUMARAN, D., MCCLELLAND, J. L., AND HILL, F. Language models show human-like content effects on reasoning. *arXiv preprint arXiv:2207.07051* 2, 3 (2022).
- [6] DU, M., TUAN, L. A., JI, B., LIU, Q., AND NG, S.-K. Mercury: A code efficiency benchmark for code large language models. *Advances in Neural Information Processing Systems* 37 (2024).
- [7] FAN, A., GOKKAYA, B., HARMAN, M., LYUBARSKIY, M., SENGUPTA, S., YOO, S., AND ZHANG, J. M. Large language models for software engineering: Survey and open problems. In *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)* (2023), IEEE, pp. 31–53.
- [8] HACKFORTH, R. *Plato: phaedrus*. No. 119. Cambridge University Press, 1972.
- [9] HAYES-ROTH, B., AND HAYES-ROTH, F. A cognitive model of planning. *Cognitive science* 3, 4 (1979), 275–310.
- [10] LAMPINEN, A. K., DASGUPTA, I., CHAN, S. C., SHEAHAN, H. R., CRESWELL, A., KUMARAN, D., MCCLELLAND, J. L., AND HILL, F. Language models, like humans, show content effects on reasoning tasks. *PNAS nexus* 3, 7 (2024), pgae233.
- [11] LEUNG, S. C. The cognitive impacts of large language model interactions on problem solving and decision making using eeg analysis. *Frontiers in Computational Neuroscience* 19 (2025), 1556483.
- [12] LI, J., TAO, C., LI, J., LI, G., JIN, Z., ZHANG, H., FANG, Z., AND LIU, F. Large language model-aware in-context learning for code generation. *ACM Transactions on Software Engineering and Methodology* 34, 7 (2025), 1–33.
- [13] LYU, M. R., RAY, B., ROYCHOUDHURY, A., TAN, S. H., AND THONGTANUNAM, P. Automatic programming: Large language models and beyond. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025), 1–33.
- [14] MADAN, A., TANDON, N., GUPTA, P., HALLINAN, S., GAO, L., WIEGREFFE, S., ALON, U., DZIRI, N., PRABHUMOYE, S., YANG, Y., ET AL. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems* 36 (2023), 46534–46594.
- [15] MUENNIGHOFF, N., LIU, Q., ZEBAZE, A., ZHENG, Q., HUI, B., ZHUO, T. Y., SINGH, S., TANG, X., VON WERRA, L., AND LONGPRE, S. Octopack: Instruction tuning code large language models. *arXiv preprint arXiv:2308.07124* (2023).
- [16] PLAAT, A., WONG, A., VERBERNE, S., BROEKENS, J., AND VAN STEIN, N. Multi-step reasoning with large language models, a survey. *ACM Computing Surveys* (2025).
- [17] PLAAT, A., WONG, A., VERBERNE, S., BROEKENS, J., VAN STEIN, N., AND BACK, T. Reasoning with large language models, a survey. *arXiv preprint arXiv:2407.11511* (2024).
- [18] PRESS, O., ZHANG, M., MIN, S., SCHMIDT, L., SMITH, N. A., AND LEWIS, M. Measuring and narrowing the compositionality gap in language models. In *Findings of the Association for Computational Linguistics: EMNLP 2023* (2023), pp. 5687–5711.
- [19] SMITHSON, M., AND VERKUILEN, J. A better lemon squeezer? maximum-likelihood regression with beta-distributed dependent variables. *Psychological methods* 11, 1 (2006), 54.
- [20] VLASTOS, G. The socratic elenchus. *The Journal of Philosophy* 79, 11 (1982), 711–714.
- [21] WANG, X., HU, Z., LU, P., ZHU, Y., ZHANG, J., SUBRAMANIAM, S., LOOMBA, A. R., ZHANG, S., SUN, Y., AND WANG, W. Scibench: Evaluating college-level scientific problem-solving abilities of large language models. *arXiv preprint arXiv:2307.10635* (2023).
- [22] WEI, J., WANG, X., SCHUURMANS, D., BOSMA, M., XIA, F., CHI, E., LE, Q. V., ZHOU, D., ET AL. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
- [23] WOHLIN, C., RUNESON, P., HÖST, M., OHLSSON, M. C., REGNELL, B., WESSLÉN, A., ET AL. *Experimentation in software engineering*, vol. 236. Springer, 2012.
- [24] XU, F., LIN, Q., HAN, J., ZHAO, T., LIU, J., AND CAMBRIA, E. Are large language models really good logical reasoners? a comprehensive evaluation and beyond. *IEEE Transactions on Knowledge and Data Engineering* (2025).
- [25] YAO, S., ZHAO, J., YU, D., DU, N., SHAFRAN, I., NARASIMHAN, K. R., AND CAO, Y. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations* (2022).
- [26] ZHUO, T. Y., VU, M. C., CHIM, J., HU, H., YU, W., WIDYASARI, R., YUSUF, I. N. B., ZHAN, H., HE, J., PAUL, I., ET AL. Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions. *arXiv preprint arXiv:2406.15877* (2024).